Gemini Connector Strategy & Agent Development Framework

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Gemini Connector Strategy & Agent Development Framework (Google) 4
What you get . . . 4
What this delivers . . . 4
The standard deployment model . . . 4
A note on preview . . . 4
Outcome . . . 5
1. Solution Overview — Gemini Connector Strategy & Agent Development Framework 5
What this is . . . 5
The two things being governed . . . 5
The governance model (how the pieces fit) . . . 6
What you can control . . . 7
Limitations (be honest with buyers) . . . 7
The standard deployment model . . . 7
2. Requirements Guide — Gemini Connector Strategy & Agent Development Framework 7
2.1 Gemini Enterprise provisioned on a Google Cloud project . . . 8
2.2 Gemini Enterprise administrative rights . . . 8
2.3 An identity provider and identity mapping . . . 8
2.4 A data-governance baseline and data-classification scheme . . . 8
2.5 A data-access boundary decision (VPC Service Controls / IAM / residency) . . . 8
2.6 The agent-development framework tooling (Agent Platform) . . . 9
2.7 Change control, pilot scope, and rollback . . . 9
Checklist . . . 9
Global Requirements — Google Workspace & Gemini Solutions 9
1. Google Workspace / Cloud Identity tenant . . . 10
2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10
3. DNS control for the routed domains . . . 10
4. Identity & provisioning source of truth . . . 11
5. A pilot scope (OU / group) and change control . . . 11
6. Security, compliance & governance stakeholders . . . 11
3. Implementation Guide — Gemini Connector Strategy & Agent Development Framework 12
Naming (set once, reuse verbatim) . . . 12
Part A — Connector strategy . . . 12
Part B — Agent-development framework (design → review → deploy → monitor) . . . 13
Step 9 — Validate on the pilot, then expand . . . 14
5. Testing Guide — Gemini Connector Strategy & Agent Development Framework 15

Test 1 — Connector is on the approved allow-list . . . 15
Test 2 — Data-access boundary present . . . 15
Test 3 — Source ACLs are respected (the core boundary test) . . . 15
Test 4 — IAM least privilege . . . 15
Test 5 — Agent registered and identity-bound . . . 16
Test 6 — Agent tools/connectors are the approved set only . . . 16
Test 7 — Agent evaluation gate passed . . . 16
Test 8 — Guardrails active (gateway + Model Armor) . . . 16
Test 9 — Agent audience is bounded . . . 16
Test 10 — Coverage-and-risk / drift report . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Gemini Connector Strategy & Agent Development Framework 17
A. Discovery / scripts return nothing (or an API error) . . . 17
B. A user sees data they should not (ACL not respected) . . . 17
C. Connector present but not approved . . . 18
D. Boundary / VPC Service Controls not detected . . . 18
E. Agent-development framework gaps . . . 18
F. Agent shared too broadly . . . 18
G. Diagnostics quick reference . . . 19
H. When to escalate . . . 19
7. Best Practices — Gemini Connector Strategy & Agent Development Framework 19
8. Deliverables Summary — Gemini Connector Strategy & Agent Development Framework 20
Documentation . . . 20
PowerShell scripts . . . 21
What the buyer receives . . . 21
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Gemini Connector Strategy & Agent Development Framework (Google)

White-label · Google

Gemini's value comes from **connectors** to enterprise data and **agents** that act on it — and both are risk surfaces if they are built ad hoc. Connect a source with the wrong permissions and Gemini surfaces data users shouldn't see; let anyone build and share an agent and capability multiplies faster than governance can follow. This solution defines the **connector strategy** and the **agent-development framework** for Gemini Enterprise so AI expands **under control** instead of becoming an ungoverned sprawl.

This is a complete, **deploy-it-today** package written for a senior administrator or architect standing up governed Gemini Enterprise connectors and agents, pilot-first and rollback-ready.

What you get

• A plain-language **Solution Overview** of the connector + agent governance model — how connectors expose data, where boundaries apply (source-ACL respect, VPC Service Controls, IAM, residency), and the agent-development guardrails.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite.

• An **Implementation Guide** of ordered steps — connector **approval criteria** and **data-access boundaries, then the agent framework across** design → review → deploy → monitor**.

• Five **PowerShell scripts** for connector/agent discovery, boundary/guardrail conformance validation, a coverage-and-risk report, safe remediation, and a scope helper.

• **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

Approved connectors on a classification-based allow-list with verified source ACLs; **data-access boundaries** that keep connectors and agents inside a VPC Service Controls perimeter, least-privilege IAM, and the source's own permissions; and a **governed agent framework** — registered, identity-bound, gateway-mediated agents that pass an evaluation gate before deploy and are monitored after — all proven on a pilot connector and agent first.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with a boundary/guardrail conformance gate and a documented rollback.

A note on preview

Gemini Enterprise (formerly Agentspace) and its Agent Platform (formerly Vertex AI Agent Builder) are **evolving and partly in preview**. Every specific in this package is **cited to Google's public documentation** or flagged **"verify in your tenant"** — nothing is invented.

Outcome

Gemini Enterprise connectors and agents that expand capability under control — approved connectors, enforced data boundaries, and a governed agent framework — instead of ad-hoc connectors, uncontrolled agents, and data over-exposure. Deployable by a single administrator and validated before you scale.

1. Solution Overview — Gemini Connector Strategy & Agent Development Framework

> ***Vendor boundary.*** *Google's* ***Gemini Enterprise*** *(formerly *Google Agentspace*) and its* ***Agent Platform*** *(formerly *Vertex AI Agent Builder*) are* ***evolving and partly in preview****. Named capabilities below are* ***cited to Google's public documentation*** *where possible; anything not confirmable against a Google doc is flagged* ***"verify in your tenant."*** *Never treat a rolling-out feature as fixed — confirm it in the Google Cloud console and the [Gemini Enterprise release notes](https://docs.cloud.google.com/gemini/enterprise/docs/release-notes).*

What this is

This solution defines the **connector strategy** and the **agent-development framework** for Gemini Enterprise: *which connectors are approved and where data-access boundaries apply*, and *a governed way to build, review, deploy, and monitor agentic workflows*. It is deliberately distinct from a general adoption/guardrails engagement — its two subjects are the **connectors that expose enterprise data** to Gemini and the **agents that act on that data**, because both multiply quickly and both are risk surfaces when built ad hoc.

The two things being governed

1. Connectors (how Gemini Enterprise exposes data)

Gemini Enterprise reaches enterprise data through **connectors** that populate **data stores** in the underlying Discovery Engine. Google documents four connector categories ([Introduction to connectors and data stores](https://docs.cloud.google.com/gemini/enterprise/docs/connectors/introduction-to-connectors-and-data-stores)):

- **Prebuilt connectors** — direct integrations to Google sources (e.g. Google Drive, Gmail, Calendar, Groups, Sites) and Google Cloud data (BigQuery, Cloud Storage, Cloud SQL, Spanner, Firestore, Bigtable, AlloyDB), plus named third-party sources (Microsoft SharePoint/OneDrive/Outlook/Entra ID, Atlassian Jira & Confluence — Cloud and Data Center, ServiceNow, Salesforce, Box, Dropbox, Slack, Notion, and more; Google states **"100+ connectors"** — the exact list and which support write **Actions** vs read-only is **verify-in-tenant**).
- **Custom connectors** — a Fetch → Transform → Sync pipeline for proprietary systems.
- **Self-hosted data sources** — customer-managed / on-prem sources reached via **Private Service Connect**.
- **Custom MCP-server data stores** — MCP-compliant internal/legacy tools and third-party systems.

Where the boundary applies. Indexed content **retains the source system's ACLs** — Gemini Enterprise *"respects source permissions,"* so a user only sees results they are authorized to see in the origin system, resolved through your identity provider ([Configure access controls](https://cloud.google.com/agentspace/agentspace-enterprise/docs/identity)). For **custom** sources you attach ACL metadata (`acl_info`) to each document so the same query returns different results per user. Around that, Google-documented boundary primitives apply: **VPC Service Controls** perimeters and **VPC firewall** egress rules, **CMEK/EKM** encryption, **data residency**, **Cloud Audit Logging**, and **IAM** roles ([security overview](https://docs.cloud.google.com/gemini/enterprise/docs/security-overview); [IAM roles](https://docs.cloud.google.com/gemini/enterprise/docs/access-control) — `discoveryengine.agentspaceAdmin`, `agentspaceUser`, `editor`, `viewer`).

2. Agents (what acts on the connected data)

Agents are built and run across a documented stack:

• **Agent Designer** — no-code/low-code builder inside Gemini Enterprise for single- and multi-step

agents ([Agent Designer overview](https://docs.cloud.google.com/gemini/enterprise/docs/agent-designer)).

• **Agent Development Kit (ADK)** — code-first framework (Python/Java/Go/TypeScript) with `adk deploy`,

and **Agent Garden** samples.

• **Agent Engine / Runtime** — managed runtime with Sessions, Memory Bank, and Code Execution

([Agent Engine overview](https://docs.cloud.google.com/vertex-ai/generative-ai/docs/agent-engine/overview)).

• **Governance components** (Gemini Enterprise Agent Platform): **Agent Registry** (approved-asset

catalog), **Agent Identity** (cryptographic per-agent id + audit trail), **Agent Gateway** (central policy-enforcement point for tool calls, with **Model Armor** prompt-injection/data-leak defenses), **Agent Security Dashboard** (Security Command Center), and **Agent Sandbox** ([Agent Platform announcement](https://cloud.google.com/blog/products/ai-machine-learning/introducing-gemini-enterprise-agent-platform)). Availability of individual components is **verify-in-tenant**.

• **Protocols: A2A** (agent-to-agent) and **MCP** (agent-to-tool) — the same connectors and tool

registry the connector strategy governs are what agents call.

The governance model (how the pieces fit)

1. **Connector approval criteria** decide *which* sources may be connected and by whom — a named

allow-list, each connector tied to a data-sensitivity classification and an owner.

1. **Data-access boundaries** decide *how far* a connected source and any agent over it can reach — ACL

respect at the source, plus VPC Service Controls, IAM least privilege, and residency around it.

1. **The agent-development framework** applies four repeatable stages to every agentic workflow —

design → **review** → **deploy** → **monitor** — so agents are registered, identity-bound, gateway-mediated, evaluated before release, and observed after.

What you can control

- **Which connectors are approved** — an allow-list mapped to data classifications and owners.
- **Data-access boundary** — source-ACL respect, VPC Service Controls perimeter, IAM roles, residency.
- **Agent lifecycle guardrails** — registry entry, agent identity, gateway/tool policy, evaluation gate, audience/sharing scope, monitoring.
- **Scope** — the Google Cloud project(s) / location the strategy and framework apply to, pilot-first.

Limitations (be honest with buyers)

- **Gemini Enterprise and the Agent Platform are evolving/preview.** Connector names, the agent governance components, console menu paths, and GA-vs-preview status **change** — every specific is cited or flagged, and must be confirmed in your tenant.
- **Respecting source ACLs depends on correct source permissions and identity mapping.** If the source's ACLs are wrong, or identity mapping is misconfigured, results can over- or under-expose — this is why boundary conformance is *tested*, not assumed.
- **Governing connectors does not clean up existing over-sharing** in the source systems; that is remediated at the source.
- This framework governs *how* agents and connectors are built and bounded; it does not replace the broader data-governance baseline, DLP, or the security stakeholders' policies it must satisfy.

The standard deployment model

> ***Assess*** *the connectors and agents in scope →* ***Configure*** *approval criteria, boundaries, and the agent framework on a pilot project →* ***Pilot*** *one approved connector and one agent →* ***Validate*** *boundary/guardrail conformance against pass/fail gates →* ***Expand*** *to more connectors and agents. Every production change is pilot-first, gated, and rollback-ready.*

2. Requirements Guide — Gemini Connector Strategy & Agent Development Framework

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-google.md`. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim across steps, scripts, and tests.

> ***Preview note.*** *Gemini Enterprise (formerly Agentspace) and the Agent Platform (formerly Vertex AI Agent Builder) are rolling out. Where a requirement names a specific role, API, or console area it is* ***cited****; confirm each in your tenant and the [release notes](https://docs.cloud.google.com/gemini/enterprise/docs/release-notes) —* ***verify in your tenant***

before relying on it.

2.1 Gemini Enterprise provisioned on a Google Cloud project

Description: Gemini Enterprise enabled on a Google Cloud **project** with the required edition/ licensing and the **Discovery Engine API** enabled. **Why:** Connectors (data stores) and agents live under this project; every approval, boundary, and framework control is scoped to it. **Verify:** `console.cloud.google.com` → confirm Gemini Enterprise is enabled and the Discovery Engine API is on; run `GEMCONN-Discovery.ps1 -ProjectId 'your-project-id'`. **Implement-if-missing:** Provision Gemini Enterprise and licenses through Google or a reseller; enable the Discovery Engine API on the project. (Edition specifics — Standard/Plus, data residency — are **verify-in-tenant**.)

2.2 Gemini Enterprise administrative rights

Description: An operating account with the Gemini Enterprise admin role (`roles/discoveryengine.agentspaceAdmin`) or equivalent, plus at least `roles/discoveryengine.viewer` for the read-only scripts ([IAM roles](https://docs.cloud.google.com/gemini/enterprise/docs/access-control)). **Why:** Approving/configuring connectors and governing agents requires admin rights; the discovery/ validation/report scripts require read access. **Verify:** Project **IAM & Admin** shows the `discoveryengine.*` bindings; `GEMCONN-Set-ScopeTargeting.ps1` lists them. **Implement-if-missing:** Grant the least-privilege `discoveryengine.*` role at the narrowest scope; role names/availability are **verify-in-tenant**.

2.3 An identity provider and identity mapping

Description: A configured **identity provider** and, for external identities, **identity mapping / Workforce Identity Federation**, so Gemini can resolve *who* is asking and honor source ACLs ([Configure identity provider](https://cloud.google.com/agentspace/docs/configure-identity-provider)). **Why:** Data-access boundaries depend on Gemini resolving the authenticated principal and matching it to each source's ACLs — "respect source permissions" only works with identity resolved. **Verify:** Confirm the identity provider is configured and external identities map correctly. **Implement-if-missing:** Configure the identity provider and identity mapping before connecting any sensitive source.

2.4 A data-governance baseline and data-classification scheme

Description: An existing data-governance baseline (source ACLs, DLP, retention) and a **classification scheme** (e.g. public / internal / confidential / restricted) to tag each connector against. **Why:** Connector approval criteria are meaningless without a classification to approve *against*; the boundary respects source ACLs, so those ACLs must be correct. **Verify:** A written classification scheme exists and the source systems' ACLs are trusted. **Implement-if-missing:** Establish the classification scheme and remediate source over-sharing before connecting a source to Gemini.

2.5 A data-access boundary decision (VPC Service Controls / IAM / residency)

Description: A decision on the **boundary** around Gemini: whether a **VPC Service Controls** perimeter applies, which **IAM** roles bound access, **VPC firewall** egress limits, and any **data- residency** requirement

([security overview](https://docs.cloud.google.com/gemini/enterprise/docs/security-overview)). **Why:** The boundary is what stops a connected source or agent from reaching further than approved. **Verify:** `GEMCONN-Validation.ps1` reports whether a perimeter and org policies are present. **Implement-if-missing:** Define/confirm a VPC Service Controls perimeter covering the project, set IAM to least privilege, and record the residency requirement. (Perimeter/residency specifics are **verify-in-tenant**.)

2.6 The agent-development framework tooling (Agent Platform)

Description: Access to the agent build/run/govern tooling in scope — **Agent Designer** and/or **ADK + Agent Engine**, and the governance components (**Agent Registry, Agent Identity, Agent Gateway, Agent Security Dashboard, Agent Sandbox**) where available ([Agent Platform](https://cloud.google.com/blog/products/ai-machine-learning/introducing-gemini-enterprise-agent-platform)). **Why:** The framework's design/review/deploy/monitor stages assume the registry, identity, gateway, and observability exist; which are enabled shapes how much of the framework is enforced by the platform vs by process. **Verify:** Confirm in the console which Agent Platform components are available on the project. **Implement-if-missing:** Enable the available components; for any not yet GA, enforce the equivalent control by process and flag it **verify-in-tenant**.

2.7 Change control, pilot scope, and rollback

Description: An approved change window, a named **pilot project/scope** (one connector + one agent), and a documented rollback before connecting a production source or exposing an agent (per `global-requirements-google.md` §5). **Why:** Connecting a source or publishing an agent can over-expose data; the pilot limits blast radius and the rollback is the safety net. **Verify:** An approved change record, the pilot scope, and a written back-out exist. **Implement-if-missing:** Capture the current state (`GEMCONN-Report.ps1 -SaveBaseline`), define the pilot, and write the rollback (disable the connector / unpublish the agent).

Checklist

- Gemini Enterprise on a project + Discovery Engine API — `GEMCONN-Discovery.ps1`
- Gemini Enterprise admin role (+ viewer for scripts) — `GEMCONN-Set-ScopeTargeting.ps1`
- Identity provider + identity mapping configured
- Data-governance baseline + classification scheme
- Data-access boundary decision (VPC Service Controls / IAM / residency) — `GEMCONN-Validation.ps1`
- Agent Platform tooling available (Designer / ADK / registry / gateway)
- Change window, pilot scope, and rollback documented

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The* ***Google Admin console*** *(admin.google.com),* ***Google Cloud / Cloud Identity****, and the* ***Microsoft / Entra*** *sides are authored at exact, current click-by-click detail from the vendors' public documentation.* ***Gemini Enterprise*** *capabilities that are in preview or rolling out are* ***cited or flagged "verify in your tenant"*** *— never invented.* ***Third-party migration tools*** *(e.g. for mail/Drive/Chat data movement) and* ***third-party enterprise apps*** *(ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — Gemini Connector Strategy & Agent Development Framework

Follow these steps **in order**. Connector work is done in the **Google Cloud console** (`https://console.cloud.google.com`, Gemini Enterprise / Agentspace admin area) and confirmed with the `GEMCONN-*` scripts; agent work is done in **Agent Designer** or **ADK + Agent Engine**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Approve → bound → build under guardrails → validate → expand. Do it on a **pilot project** (one connector, one agent) first. Keep a rollback for every change.* ***Preview note:*** *exact console menu paths and the GA-vs-preview status of Agent Platform components are **verify-in-tenant** — the steps name the documented area and tell you to confirm it in your console.*

Naming (set once, reuse verbatim)

- Connector approval record: `Connector Approval – Pilot Project – Prod`
- Data-access boundary (perimeter): `Data Access Boundary – Pilot Project – Prod`
- Agent under development: `Agent Build – Pilot Project – Design`
- Agent released: `Agent Build – Pilot Project – Prod`

Part A — Connector strategy

Step 1 — Confirm scope and inventory what exists

1. Run `GEMCONN-Set-ScopeTargeting.ps1 -ProjectId 'your-project-id'` to confirm the project, the

Discovery Engine resource-path prefix, the Gemini Enterprise IAM bindings, and whether a boundary exists.

1. Run `GEMCONN-Discovery.ps1 -ProjectId 'your-project-id'` for the current connectors (data stores) and

agents. Treat any empty result as **verify-in-tenant** (preview surface), not "none".

Step 2 — Define the connector approval criteria

1. Write the **approval criteria** each connector must meet before it is connected. A workable baseline:

- a **named business owner** and use case;
- a **data classification** (public / internal / confidential / restricted) for the source, from the

scheme in Requirements §2.4;

- **source ACLs verified correct** (Gemini respects them — bad ACLs over-expose);
- **identity mapping confirmed** for the source's users

([identity](https://cloud.google.com/agentspace/agentspace-enterprise/docs/identity));

- a **sync cadence** decision (full/incremental/identity — Google documents identity sync as often as

every 30 minutes).

1. Record each approved connector in the allow-list `Connector Approval – Pilot Project – Prod` (one

data-store id per line — the format `GEMCONN-Validation.ps1 -ApprovedConnectorsPath` consumes).

Step 3 — Set the data-access boundary

1. Confirm/define a **VPC Service Controls** perimeter covering the project

(`Data Access Boundary – Pilot Project – Prod`) so connectors and agents cannot exfiltrate beyond it ([security overview](https://docs.cloud.google.com/gemini/enterprise/docs/security-overview); [VPC-SC with Agent Platform](https://docs.cloud.google.com/gemini-enterprise-agent-platform/machine-learning/general/vpc-service-controls)).

1. Set **IAM** to least privilege — grant `discoveryengine.agentspaceAdmin` only to admins,

`agentspaceUser` to users who need it, and use **app-level IAM** to restrict users to specific apps ([app access controls](https://docs.cloud.google.com/gemini/enterprise/docs/iam-policy-for-apps)).

1. Record the **residency** requirement and confirm **VPC firewall** egress limits where required.

2. Run `GEMCONN-Validation.ps1 -ProjectId 'your-project-id' -ApprovedConnectorsPath .\approved-connectors.txt`

— confirms boundary present, org policies present, and connectors on the allow-list.

Step 4 — Connect the pilot connector

1. In the Gemini Enterprise / Agentspace admin area of the console, create the **data store** for the

one approved pilot source (prebuilt, custom, self-hosted via Private Service Connect, or custom MCP server as applicable). *Confirm the exact menu path in your console — it is verify-in-tenant.*

1. Run the initial sync and confirm **identity sync** is on so ACLs propagate.

2. Run `GEMCONN-Discovery.ps1` — the new connector should appear; run `GEMCONN-Report.ps1 -SaveBaseline`

to capture the pre-expansion baseline.

Part B — Agent-development framework (design → review → deploy → monitor)

Step 5 — Design (register + identity)

1. Build the pilot agent (`Agent Build – Pilot Project – Design`) in **Agent Designer** (no-code) or

ADK (code-first) ([Agent Designer](https://docs.cloud.google.com/gemini/enterprise/docs/agent-designer)).

1. Register it in the **Agent Registry** so only governed, approved agents/tools are discoverable, and

confirm it gets an **Agent Identity** (cryptographic id → audit trail) ([Agent Platform](https://cloud.google.com/blog/products/ai-machine-learning/introducing-gemini-enterprise-agent-platform)). *If a component is not yet available in your tenant, enforce the equivalent by process and flag it.*

1. Restrict the tools/connectors the agent may call to the **approved** set (tool governance / Agent Gateway); the same allow-list from Step 2 governs what the agent can reach.

Step 6 — Review (evaluate before release)

1. Test the agent in **Preview / playground** and run **Gen AI Evaluation** — AutoRaters, a User Simulator, and online evaluation ([monitor an agent](https://docs.cloud.google.com/agent-builder/agent-engine/manage/monitoring)).
1. Confirm **guardrails**: **Agent Gateway** policy + **Model Armor** (prompt-injection / data-leak defense) apply, and boundary tests (Testing Guide) pass — an agent must not surface data the caller is not entitled to.
1. This is the **review gate**: do not deploy until evaluation and boundary conformance pass.

Step 7 — Deploy (bounded audience)

1. Deploy via **Agent Engine** (or publish in Gemini Enterprise) as `Agent Build – Pilot Project – Prod`.
2. **Share only to the approved audience** — if the admin enables sharing, owners can share to Google Identity groups; never leave an agent shared broadly ([share an agent](https://docs.cloud.google.com/gemini/enterprise/docs/agent-designer/share-agent)).
1. Run `GEMCONN-Validation.ps1` — confirm the agent's audience is within scope (audience read is preview / verify-in-tenant; the script prompts a console check).

Step 8 — Monitor

1. Use **Agent Engine observability** — dashboards for token consumption, latency, error rates, and tool calls, plus OpenTelemetry traces (Cloud Trace/Monitoring/Logging) ([monitor an agent](https://docs.cloud.google.com/agent-builder/agent-engine/manage/monitoring)).
1. Watch the **Agent Security Dashboard** (Security Command Center) for anomaly/threat detection where available.
1. Run `GEMCONN-Report.ps1 -BaselinePath .\gemconn-baseline.json` each cycle to track connector/agent coverage-and-risk drift.

Step 9 — Validate on the pilot, then expand

1. Run the **Testing Guide** — boundary/guardrail conformance must pass on the pilot connector and agent.
2. Only then **expand**: approve the next connectors against the same criteria, connect them, and build the next agents through the same design→review→deploy→monitor stages.

1. Re-validate each cycle with `GEMCONN-Validation.ps1` and track drift with `GEMCONN-Report.ps1`.

Order of operations

`Confirm scope → define approval criteria → set boundary → connect pilot connector → design agent (register + identity) → review (evaluate + guardrails gate) → deploy (bounded audience) → monitor → validate → expand`

5. Testing Guide — Gemini Connector Strategy & Agent Development Framework

Run after the pilot connector is connected and the pilot agent reaches the **review gate** (Implementation Steps 4 and 6), again after deploy (Step 7), and after each expansion cycle. These are **boundary and guardrail conformance gates** — an agent or connector must not surface data a caller is not entitled to. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

> ***Preview note.*** *Some agent audience/registry surfaces are rolling out; where a test cannot be read via a stable API, perform it in the console and record the result — **verify in your tenant** rather than assume a pass.*

Test 1 — Connector is on the approved allow-list

Do: Run `GEMCONN-Validation.ps1 -ProjectId 'your-project-id' -ApprovedConnectorsPath .\approved-connectors.txt`. **Expected:** Every connected data store appears on the allow-list; no unapproved connector is present.

Test 2 — Data-access boundary present

Do: Run `GEMCONN-Validation.ps1`; or check **VPC Service Controls** and org policies in the console. **Expected:** A service perimeter (`Data Access Boundary – Pilot Project – Prod`) and org policy constraints cover the project — **PASS**; missing boundary is **WARN/fail**.

Test 3 — Source ACLs are respected (the core boundary test)

Do: As **User A** (entitled to a document in the source) and **User B** (not entitled), issue the same Gemini Enterprise query that would surface that document. **Expected:** User A sees it; **User B does not** — Gemini respects the source ACL. If User B sees it, identity mapping or the source ACL is wrong — fix before proceeding ([identity/access control](https://cloud.google.com/agentspace/agentspace-enterprise/docs/identity)).

Test 4 — IAM least privilege

Do: Run `GEMCONN-Set-ScopeTargeting.ps1`; review the `discoveryengine.*` bindings. **Expected:** `agentspaceAdmin` is held only by admins; users hold `agentspaceUser`; no unexpected broad grants — least privilege confirmed.

Test 5 — Agent registered and identity-bound

Do: Confirm the pilot agent is in the **Agent Registry** and has an **Agent Identity**; or note the console state. **Expected:** The agent is a governed, registered asset with a unique identity and audit trail — **PASS**. *(If the registry/identity component is not yet available, record the process-enforced equivalent — verify-in-tenant.)*

Test 6 — Agent tools/connectors are the approved set only

Do: Inspect the agent's allowed tools/connectors (Agent Gateway / tool governance). **Expected:** The agent can call **only** approved connectors from the Step 2 allow-list — no unapproved source or tool is reachable.

Test 7 — Agent evaluation gate passed

Do: Run **Gen AI Evaluation** (AutoRaters / User Simulator / online evaluation) and review results ([monitor an agent](https://docs.cloud.google.com/agent-builder/agent-engine/manage/monitoring)). **Expected:** The agent meets the agreed quality/safety thresholds — the **review gate** passes before deploy. A failing evaluation blocks deployment.

Test 8 — Guardrails active (gateway + Model Armor)

Do: Submit a prompt-injection / data-exfiltration probe to the agent. **Expected:** The **Agent Gateway** policy and **Model Armor** block or neutralize it; the attempt is logged — the guardrail holds.

Test 9 — Agent audience is bounded

Do: Confirm the deployed agent (`Agent Build – Pilot Project – Prod`) is shared only to its approved Google Identity group; `GEMCONN-Validation.ps1` prompts this console check. **Expected:** No agent is public or shared beyond its approved audience.

Test 10 — Coverage-and-risk / drift report

Do: Run `GEMCONN-Report.ps1 -BaselinePath .\gemconn-baseline.json`. **Expected:** A clear coverage-and-risk report — connectors by type, agent count, unapproved connectors, boundary present, a risk tally, and drift versus the saved baseline.

Pass/fail gate

Gate	Requirement
Connector pass	Tests 1–4 pass — connector approved, boundary present, **source ACLs respected**, IAM least privilege
Agent review gate	Tests 5–8 pass — registered + identity-bound, approved tools only, evaluation passed, guardrails active — *before* deploy
Deploy pass	Test 9 passes — audience bounded

Gate	Requirement
Expand pass	Tests 1–10 pass on the pilot before approving/connecting the next connector or agent

6. Troubleshooting Guide — Gemini Connector Strategy & Agent Development Framework

Use `GEMCONN-Set-ScopeTargeting.ps1` / `GEMCONN-Discovery.ps1` for scope and inventory, `GEMCONN-Validation.ps1` for boundary/guardrail conformance, `GEMCONN-Report.ps1` for coverage-and-risk and drift, and `GEMCONN-Remediation.ps1` (defaults to `-WhatIf`) for safe fixes. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

> ***Preview note.*** *Gemini Enterprise / Agentspace and the Agent Platform are rolling out. If a script returns empty or errors on an API version, that often means the surface differs in your tenant — treat it as* ***verify-in-tenant*** *and confirm in the console before concluding a control is absent.*

A. Discovery / scripts return nothing (or an API error)

Symptom	Cause	Remediation
`GEMCONN-Discovery.ps1` lists no connectors/agents	Discovery Engine API not enabled, wrong location, or preview API version	Enable the Discovery Engine API; try `-Location 'global'` (or your region) and `-ApiVersion v1alpha`; confirm in the console — **verify-in-tenant** before concluding "none"
`Could not obtain a gcloud access token`	Not authenticated	Run `gcloud auth login` and `gcloud config set project your-project-id`, then re-run
403 / permission denied	Operating account lacks `discoveryengine.viewer` (or higher)	Grant the least-privilege `discoveryengine.*` role at the project scope; re-run `GEMCONN-Set-ScopeTargeting.ps1`

B. A user sees data they should not (ACL not respected)

• This is the highest-priority failure. Gemini **respects source permissions**, so if an unentitled user sees a result, either the **source ACL is wrong** or **identity mapping is misconfigured**.

• Verify the source system's ACL for that document; verify the identity provider and identity mapping ([identity/access control](https://cloud.google.com/agentspace/agentspace-enterprise/docs/identity)).

• For **custom** sources, confirm each document carries correct `acl_info` metadata.

• Until fixed, **disable the connector** (`GEMCONN-Remediation.ps1 -Action DisableConnector`) so it stops

exposing the source. Re-run Testing Guide Test 3 before reconnecting.

C. Connector present but not approved

- `GEMCONN-Validation.ps1` flags a data store not on the allow-list. Either add it to the approval record

`Connector Approval – Pilot Project – Prod` after it meets the criteria, or disable it (`GEMCONN-Remediation.ps1 -Action DisableConnector`, default `-WhatIf` first).

D. Boundary / VPC Service Controls not detected

Symptom	Cause	Remediation
Validation reports no perimeter	No perimeter, or the account can't read Access Context Manager	Confirm/define a perimeter covering the project; grant read on the access policy; **verify-in-tenant** whether an org-level perimeter already applies
Connector egress blocked unexpectedly	VPC firewall / perimeter is doing its job (or too tight)	Confirm the source FQDN is allowed; adjust the egress rule under change control — do not remove the perimeter

E. Agent-development framework gaps

Symptom	Cause	Remediation
No Agent Registry / Identity / Gateway visible	Component not yet available in your tenant (preview)	Enforce the equivalent by process (register agents in a tracked list, restrict tools manually) and flag **verify-in-tenant**; enable the component when it reaches your tenant
Agent surfaces unentitled data	Agent reaching an unapproved connector, or ACL issue (see B)	Restrict the agent's tools to the approved allow-list (Agent Gateway / tool governance); fix the underlying ACL
Evaluation gate skipped	Agent deployed without Gen AI Evaluation	Roll back the deploy; run evaluation + boundary tests; only redeploy after the review gate passes

F. Agent shared too broadly

- `GEMCONN-Validation.ps1` prompts a console audience check (audience read is preview). If an agent is

public or shared beyond its group, narrow the audience in the console, or `GEMCONN-Remediation.ps1 -Action UnpublishAgent` (default `-WhatIf`). Automated unpublish is best-effort on the preview surface — the script tells you the console step if the API path is unavailable.

G. Diagnostics quick reference

```
gcloud auth print-access-token                                    # confirm auth
gcloud config get-value project                                   # confirm target project
gcloud projects get-iam-policy your-project-id --format=json      # Gemini Enterprise IAM bindings
gcloud access-context-manager perimeters list --format=json       # VPC Service Controls perimeters
.\GEMCONN-Discovery.ps1 -ProjectId 'your-project-id' -ApiVersion v1alpha   # try preview API version
```

H. When to escalate

If the Discovery Engine API is enabled, the account holds the right `discoveryengine.*` role, the API version matches your tenant, and connectors/agents still do not appear or ACLs still misbehave after verifying identity mapping and source permissions, collect `GEMCONN-Report.ps1` output and the project/location details and open a Google Cloud support case. Cross-check the [Gemini Enterprise release notes](https://docs.cloud.google.com/gemini/enterprise/docs/release-notes) for a preview-status or API change affecting the surface.

7. Best Practices — Gemini Connector Strategy & Agent Development Framework

Field-proven practices for expanding Gemini Enterprise capability **under control** — approved connectors, enforced boundaries, and a governed agent framework — rather than an ungoverned sprawl.

- **Approve connectors against a classification, not a request.** Every connector gets a named owner, a

data classification, and verified source ACLs before it is connected. An allow-list you can validate against (`GEMCONN-Validation.ps1 -ApprovedConnectorsPath`) is the difference between a strategy and a pile of ad-hoc integrations.

- **Trust the source ACL — but verify it.** Gemini *respects source permissions*, so a connected source

is only as safe as its own ACLs and your identity mapping. Test with an entitled and an unentitled user (Testing Guide Test 3) on every connector; bad ACLs over-expose the moment they are indexed.

- **Bound before you build.** Put the **VPC Service Controls** perimeter, IAM least privilege, and

residency in place *before* connecting a production source or exposing an agent — the boundary is what stops a connector or agent reaching further than approved.

- **Least privilege on the Gemini roles.** Grant `agentspaceAdmin` sparingly, `agentspaceUser` to those

who need it, and use app-level IAM to scope users to specific apps. Admin is not a default.

- **Register every agent; bind an identity.** An agent that isn't in the **Agent Registry** with an

Agent Identity is an ungoverned actor with no audit trail. Registration + identity is the entry price for any agent that touches enterprise data.

• **Gate on evaluation, always.** Run **Gen AI Evaluation** and boundary conformance *before* deploy — a

no-code agent is as capable of leaking data as a coded one. Design → **review (gate)** → deploy → monitor; never skip the review gate.

• **Route tool calls through the gateway.** Let the **Agent Gateway** (with **Model Armor**) mediate what

tools and connectors an agent may call, restricted to the same approved allow-list — one policy point, not per-agent trust.

• **Share to a group, never to everyone.** Deploy agents to a named Google Identity group audience; a

broadly shared agent is the fastest path to accidental exposure (Testing Guide Test 9).

• **Monitor after deploy.** Watch token consumption, latency, error rates, tool calls, and traces in

Agent Engine observability, and the Agent Security Dashboard for anomalies. An agent's risk is a living thing, not a launch-day checkbox.

• **Pilot one connector and one agent, then expand.** Prove the whole model — approval, boundary,

design→review→deploy→monitor — on a narrow pilot before scaling; the same repeatable pattern is what keeps expansion governed.

• **Treat every preview specific as verify-in-tenant.** Connector names, Agent Platform components, and

console paths are evolving. Cite Google's docs, confirm in your console and the release notes, and never build a control on an assumption about a rolling-out feature.

• **Keep a rollback for every change.** Disabling a connector (`GEMCONN-Remediation.ps1 -Action

DisableConnector`) or unpublishing an over-shared agent must be a written, ready step before you make the change.

8. Deliverables Summary — Gemini Connector Strategy & Agent Development Framework

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The connector + agent governance model — how connectors expose data, where boundaries apply, and the agent-development guardrails; benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Google)	Category-wide prerequisites, published once
3. Implementation Guide	Ordered steps — approval criteria, data-access boundary, connect connector, and the agent framework (design → review → deploy → monitor)

Document	Purpose
5. Testing Guide	Boundary/guardrail conformance gates with pass/fail criteria
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Governed AI expansion — approved connectors, enforced boundaries, gated agents

PowerShell scripts

Script	What it does
`GEMCONN-Discovery.ps1`	Read-only inventory of connectors (data stores) and agents across a project/location
`GEMCONN-Validation.ps1`	Boundary/guardrail conformance — access boundary present, org policies, connectors on the approved allow-list, agent audience check
`GEMCONN-Report.ps1`	Connector/agent coverage-and-risk report with drift versus a saved baseline
`GEMCONN-Remediation.ps1`	Safe remediation (defaults to `-WhatIf`) — disable an unapproved connector or unpublish an over-shared agent
`GEMCONN-Set-ScopeTargeting.ps1`	Resolves and confirms the project / location / IAM scope reused by every step

> *All scripts are **read-only where they read live systems**; the two change-capable actions (`GEMCONN-Remediation.ps1` and any scope change) default to `-WhatIf`. Gemini Enterprise / Agentspace surfaces are **preview/evolving** — scripts cite Google's documented APIs and flag empty or unreadable results as **verify-in-tenant** rather than asserting "none".*

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**

— pilot-first, gated, and rollback-ready.

Outcome

Gemini Enterprise connectors and agents that **expand capability under control** — approved connectors, enforced data-access boundaries, and a governed agent framework (design → review → deploy → monitor) — not an ungoverned sprawl. Proven on a pilot connector and agent before you scale, with every preview-stage specific cited or flagged for confirmation in your tenant.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GEMCONN-Discovery.ps1

Purpose: Read-only discovery for the "Gemini Connector Strategy & Agent Development Framework" solution. For a Google Cloud project and location, lists the Discovery Engine / Agentspace data stores (the "connectors" that expose enterprise data to Gemini Enterprise) with their type and content config, and enumerates the agents / assistants configured in the same scope. It calls the Google Discovery Engine REST API with a gcloud access token and makes no changes. Gemini Enterprise / Agentspace surfaces are evolving (preview) — resource shapes and API versions may differ in your tenant, so treat any empty result as 'verify in your tenant', not 'none'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GEMCONN-Discovery.ps1 -ProjectId 'your-project-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Discovery.ps1 -ProjectId 'your-project-id' -Location 'global' Requires the Google Cloud SDK (gcloud) authenticated with a principal that can read Discovery Engine resources (for example the Discovery Engine Viewer role). Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. Needs the Discovery Engine API enabled on the project. EXPECTED RESULTS A per-project snapshot: each data-store / connector with id, type and content config, and each configured agent / assistant — a single inventory of what exposes data and what acts on it, before you apply approval criteria and agent-development guardrails. Empty sections print a 'verify in your tenant' note because the APIs are in preview. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Gemini Enterprise / Agentspace are preview/rolling-out; API versions, resource names, and console paths are cited from Google's public docs where possible and otherwise flagged 'verify in tenant'. Never invents connector or agent specifics. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1')

How to run: powershell -ExecutionPolicy Bypass -File .\GEMCONN-Discovery.ps1 -ProjectId 'your-project-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Discovery.ps1 -ProjectId 'your-project-id' -Location 'global' Requires the Google Cloud SDK (gcloud) authenticated with a principal that can read Discovery Engine resources (for example the Discovery Engine Viewer role). Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. Needs the Discovery Engine API enabled on the project. EXPECTED RESULTS A per-project snapshot: each data-store / connector with id, type and content config, and each configured agent / assistant — a single inventory of what exposes data and what acts on it, before you apply approval criteria and agent-development guardrails. Empty sections print a 'verify in your tenant' note because the APIs are in preview. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Gemini Enterprise / Agentspace are preview/rolling-out; API versions, resource names, and console paths are cited from Google's public docs where possible and otherwise flagged 'verify in tenant'. Never invents connector or agent specifics. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1')

GEMCONN-Remediation.ps1

Purpose: Safe, targeted fixes for the "Gemini Connector Strategy & Agent Development Framework" solution. Two actions: 'DisableConnector' — delete / disable a named Discovery Engine data store that failed the approval criteria, so it no longer exposes its source to Gemini Enterprise; 'UnpublishAgent' — narrow or remove a named agent/engine that is shared beyond its approved audience. DEFAULTS TO

-WhatIf — it prints the change it would make and changes nothing until you pass -Execute. Both actions are destructive to the named resource only; run GEMCONN-Report.ps1 first to capture the pre-change state and confirm the id. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GEMCONN-Remediation.ps1 -Action DisableConnector -ProjectId 'your-project-id' -ResourceId 'your-datastore-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Remediation.ps1 -Action UnpublishAgent -ProjectId 'your-project-id' -ResourceId 'your-engine-id' Apply for real (explicit): ... -Execute Requires the Google Cloud SDK (gcloud) authenticated with a principal that can edit Discovery Engine resources (an editor/admin-level role). Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. Agent audience controls are a preview surface — UnpublishAgent is best-effort and may require a console step. EXPECTED RESULTS In -WhatIf mode (default): a printed description of the connector disable or agent unpublish — nothing changes. With -Execute: the delete/unpublish call is issued and the result reported. .NOTES White-label — the repeatable way. Defaults to -WhatIf. Read-only until you pass -Execute. Acts only on the single resource id you name. Gemini Enterprise agent audience controls are preview; where no stable API exists, the script tells you the console step to take instead of guessing. Never invents API or resource specifics. #> [CmdletBinding()] param([Parameter(Mandatory)] [ValidateSet('DisableConnector','UnpublishAgent')] [string]$Action, [Parameter(Mandatory)] [string]$ProjectId, [Parameter(Mandatory)] [string]$ResourceId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1', [switch]$Execute # omit for -WhatIf (default safe mode))

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GEMCONN-Remediation.ps1 -Action DisableConnector -ProjectId 'your-project-id' -ResourceId 'your-datastore-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Remediation.ps1 -Action UnpublishAgent -ProjectId 'your-project-id' -ResourceId 'your-engine-id' Apply for real (explicit): ... -Execute Requires the Google Cloud SDK (gcloud) authenticated with a principal that can edit Discovery Engine resources (an editor/admin-level role). Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. Agent audience controls are a preview surface — UnpublishAgent is best-effort and may require a console step. EXPECTED RESULTS In -WhatIf mode (default): a printed description of the connector disable or agent unpublish — nothing changes. With -Execute: the delete/unpublish call is issued and the result reported. .NOTES White-label — the repeatable way. Defaults to -WhatIf. Read-only until you pass -Execute. Acts only on the single resource id you name. Gemini Enterprise agent audience controls are preview; where no stable API exists, the script tells you the console step to take instead of guessing. Never invents API or resource specifics. #> [CmdletBinding()] param([Parameter(Mandatory)] [ValidateSet('DisableConnector','UnpublishAgent')] [string]$Action, [Parameter(Mandatory)] [string]$ProjectId, [Parameter(Mandatory)] [string]$ResourceId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1', [switch]$Execute # omit for -WhatIf (default safe mode))

GEMCONN-Report.ps1

Purpose: Read-only reporting for the "Gemini Connector Strategy & Agent Development Framework" solution. Produces a coverage-and-risk snapshot for a Google Cloud project: how many connectors (data stores) exist and their types, how many agents exist, which connectors carry no approval note, whether an access boundary is present, and a simple risk tally (unapproved connectors + boundary gaps). If -BaselinePath points to a JSON file from a previous run it reports the DRIFT (new connectors / new agents / risk-tally change); with -SaveBaseline it writes this run for next-cycle comparison. Reads live state via gcloud and the Discovery Engine REST API; makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GEMCONN-Report.ps1 -ProjectId 'your-project-id' powershell

-ExecutionPolicy Bypass -File .\GEMCONN-Report.ps1 -ProjectId 'your-project-id' -BaselinePath .\gemconn-baseline.json -SaveBaseline Requires the Google Cloud SDK (gcloud) authenticated with viewer access to Discovery Engine and Access Context Manager. Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. EXPECTED RESULTS A coverage-and-risk report — connector count by type, agent count, connectors lacking an approval record, boundary present yes/no, and a risk tally. With -BaselinePath, a drift line comparing this run to the saved baseline. Preview surfaces that cannot be read are shown as 'verify in tenant', not zero. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Gemini Enterprise connector/agent surfaces are preview; counts reflect what the API returns in your tenant. Boundary read from documented gcloud commands. Never invents connector/agent specifics. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1', [string]$ApprovedConnectorsPath, [string]$BaselinePath, [switch]$SaveBaseline)

How to run: powershell -ExecutionPolicy Bypass -File .\GEMCONN-Report.ps1 -ProjectId 'your-project-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Report.ps1 -ProjectId 'your-project-id' -BaselinePath .\gemconn-baseline.json -SaveBaseline Requires the Google Cloud SDK (gcloud) authenticated with viewer access to Discovery Engine and Access Context Manager. Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. EXPECTED RESULTS A coverage-and-risk report — connector count by type, agent count, connectors lacking an approval record, boundary present yes/no, and a risk tally. With -BaselinePath, a drift line comparing this run to the saved baseline. Preview surfaces that cannot be read are shown as 'verify in tenant', not zero. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Gemini Enterprise connector/agent surfaces are preview; counts reflect what the API returns in your tenant. Boundary read from documented gcloud commands. Never invents connector/agent specifics. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1', [string]$ApprovedConnectorsPath, [string]$BaselinePath, [switch]$SaveBaseline)

GEMCONN-Set-ScopeTargeting.ps1

Purpose: Scope helper for the "Gemini Connector Strategy & Agent Development Framework" solution. Resolves the target governance scope from a Google Cloud project id (and location), prints the Discovery Engine resource-path prefix that connector/agent operations use, lists the current Gemini Enterprise IAM bindings on the project (who is admin/user/editor/viewer), and reports whether an access boundary (VPC Service Controls perimeter) covers it. Read-only — it confirms WHERE approval criteria and agent guardrails will apply before you assign them. Reuse the printed project/location/scope verbatim as input to the other GEMCONN scripts and the console. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GEMCONN-Set-ScopeTargeting.ps1 -ProjectId 'your-project-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Set-ScopeTargeting.ps1 -ProjectId 'your-project-id' -Location 'us' Requires the Google Cloud SDK (gcloud) authenticated with permission to read the project IAM policy and Access Context Manager. Run 'gcloud auth login' first. EXPECTED RESULTS The resolved project + location, the Discovery Engine resource-path prefix, the Gemini Enterprise IAM role bindings on the project (admin / user / editor / viewer), and whether a VPC Service Controls perimeter is present — the confirmed target you reuse verbatim in every approval decision, script, and test. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Object naming convention: Purpose - Scope - Stage, e.g. 'Connector Approval - Pilot Project - Prod'. Gemini Enterprise IAM roles (discoveryengine.agentspaceAdmin / agentspaceUser / editor / viewer) are cited from Google's public IAM docs; confirm role availability in your tenant. Never invents role or resource names. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global')

How to run: powershell -ExecutionPolicy Bypass -File .\GEMCONN-Set-ScopeTargeting.ps1 -ProjectId 'your-project-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Set-ScopeTargeting.ps1 -ProjectId 'your-project-id' -Location 'us' Requires the Google Cloud SDK (gcloud) authenticated with permission to read the project IAM policy and Access Context Manager. Run 'gcloud auth login' first. EXPECTED RESULTS The resolved project + location, the Discovery Engine resource-path prefix, the Gemini Enterprise IAM role bindings on the project (admin / user / editor / viewer), and whether a VPC Service Controls perimeter is present — the confirmed target you reuse verbatim in every approval decision, script, and test. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Object naming convention: Purpose - Scope - Stage, e.g. 'Connector Approval - Pilot Project - Prod'. Gemini Enterprise IAM roles (discoveryengine.agentspaceAdmin / agentspaceUser / editor / viewer) are cited from Google's public IAM docs; confirm role availability in your tenant. Never invents role or resource names. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global')

GEMCONN-Validation.ps1

Purpose: Read-only conformance validation for the "Gemini Connector Strategy & Agent Development Framework" solution. For a Google Cloud project it checks the boundary and guardrail controls the framework requires: that VPC Service Controls / an access boundary is present, that org policies restricting the project exist, that connectors are inside the approved allow-list, and that agents are not published beyond their approved audience. It reads live state via gcloud and the Discovery Engine REST API and makes no changes. Where a control cannot be read because the surface is in preview, it prints WARN with a 'verify in your tenant' note rather than asserting a pass. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GEMCONN-Validation.ps1 -ProjectId 'your-project-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Validation.ps1 -ProjectId 'your-project-id' -ApprovedConnectorsPath .\approved-connectors.txt Requires the Google Cloud SDK (gcloud) authenticated with read access to Access Context Manager, org policies, and Discovery Engine (viewer-level roles). Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. EXPECTED RESULTS A short PASS/WARN report: access-boundary (VPC Service Controls) present, org policy constraints present, connectors within the approved allow-list, and agents within their approved audience — the go/no-go gate before you pilot an agent against production data. Preview-gated checks report WARN with a verify-in-tenant note, never a false PASS. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Boundary primitives (VPC Service Controls, Access Context Manager, org policies) are GA and read from documented gcloud commands; Gemini Enterprise connector/agent audience controls are preview and flagged 'verify in tenant'. Never invents API or role names. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1', [string]$ApprovedConnectorsPath # optional text file: one approved data-store id per line)

How to run: powershell -ExecutionPolicy Bypass -File .\GEMCONN-Validation.ps1 -ProjectId 'your-project-id' powershell -ExecutionPolicy Bypass -File .\GEMCONN-Validation.ps1 -ProjectId 'your-project-id' -ApprovedConnectorsPath .\approved-connectors.txt Requires the Google Cloud SDK (gcloud) authenticated with read access to Access Context Manager, org policies, and Discovery Engine (viewer-level roles). Run 'gcloud auth login' and 'gcloud config set project your-project-id' first. EXPECTED RESULTS A short PASS/WARN report: access-boundary (VPC Service Controls) present, org policy constraints present, connectors within the approved allow-list, and agents within their approved audience — the go/no-go gate before you pilot an agent against production data. Preview-gated checks report WARN with a verify-in-tenant note, never a false PASS. .NOTES White-label — the repeatable way. Read-only where it reads live systems. Boundary primitives (VPC Service Controls, Access Context

Manager, org policies) are GA and read from documented gcloud commands; Gemini Enterprise connector/agent audience controls are preview and flagged 'verify in tenant'. Never invents API or role names. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$ProjectId, [string]$Location = 'global', [ValidateSet('v1','v1alpha','v1beta')] [string]$ApiVersion = 'v1', [string]$ApprovedConnectorsPath # optional text file: one approved data-store id per line)